

S2S-02: Discovery and Configuration Export

Series: S2S | **Notebook:** 2 of 12 | **Created:** March 2026 | **Last Updated:** 03/23/2026

Overview

Before migrating anything, you need a complete inventory of what exists in the source tenant. This notebook covers discovery queries, Monaco bulk export, and Terraform state bootstrapping.

Table of Contents

1. [Entity Discovery](#)
 2. [Configuration Inventory](#)
 3. [Monaco Bulk Export](#)
 4. [Terraform Bootstrap](#)
 5. [Export Validation](#)
-

Prerequisites

Requirement	Details
Source Tenant	Admin access with <code>settings.read</code> , <code>ReadConfig</code> scopes
Monaco CLI	v2.x installed (<code>monaco version</code>)
Terraform	v1.5+ with Dynatrace provider
Git Repository	Initialized repo for storing exported configuration

1. Entity Discovery

Host Inventory

Run these queries against the source tenant to understand your monitoring footprint:

```
// Host inventory by cloud provider and OS
fetch dt.entity.host
| fieldsAdd provider = if(isNotNull(awsNameTag), "AWS",
    if(isNotNull(azureResourceGroupName), "Azure",
    if(isNotNull(gcpProjectId), "GCP", "On-Premises")))
| summarize count = count(), by:{provider, osType}
| sort count desc
```

```
// Kubernetes cluster inventory
fetch dt.entity.kubernetes_cluster
| fields entity.name, id
| sort entity.name asc
```

```
// Service inventory by technology
fetch dt.entity.service
| fieldsAdd tech = serviceType
| summarize count = count(), by:{tech}
| sort count desc
```

2. Configuration Inventory

Settings 2.0 Inventory

Query the Settings API to count configuration objects by schema:

```
// Audit log: recent configuration changes (what's actively managed)
fetch logs, from:-30d
| filter matchesPhrase(log.source, "audit")
| filter contains(content, "settings")
| parse content, "JSON:json"
| summarize changes = count(), by:{json["schemaId"]}
| sort changes desc
| limit 20
```

Configuration Count Reference

Gen2 (Classic) Configuration

Category	API	Typical Range
Management zones	Config API v1: <code>/managementZones</code>	5-30
Auto-tags	Config API v1: <code>/autoTags</code> or Settings 2.0	10-50
Alerting profiles	Config API v1: <code>/alertingProfiles</code>	5-20
Notification rules	Config API v1: <code>/notifications</code> or Settings 2.0	5-50
Calculated service metrics	Config API v1: <code>/calculatedMetrics/service</code>	10-50
Request attributes	Config API v1: <code>/requestAttributes</code> or Settings 2.0	5-30
Custom services	Config API v1: <code>/service/customServices</code>	5-20
Application detection rules	Config API v1: <code>/applicationDetectionRules</code>	5-20
Conditional naming rules	Config API v1: <code>/conditionalNaming</code>	5-15
Process group detection rules	Config API v1 / Settings 2.0	5-15
Maintenance windows	Config API v1: <code>/maintenanceWindows</code> or Settings 2.0	3-10
Classic dashboards	Dashboard API v1: <code>/dashboards</code>	20-200
Credential vault	Config API v1: <code>/credentials</code>	5-20

Gen3 (Grail) Configuration

Category	API	Typical Range
Notification rules (Gen3)	Settings API: <code>builtin:alerting.profile</code>	5-50
SLO definitions	SLO API	10-100
Dashboards	Document API	20-200
Synthetic monitors	Synthetic API	10-100
Management zones	Settings API: <code>builtin:management-zones</code>	5-30
Auto-tags	Settings API: <code>builtin:tags.auto-tagging</code>	10-50
Request attributes	Settings API: <code>builtin:request-attributes</code>	5-30
Calculated metrics	Calculated metrics API	10-50
Enrichment rules	Settings API: <code>builtin:kubernetes.metadata.enrichment</code>	1-20
OpenPipeline rules	OpenPipeline API	1-10
Workflows	Automation API	5-30

3. Monaco Bulk Export

Monaco's `download` command exports your entire tenant configuration in one operation:

```

# Create manifest for source tenant
cat > manifest.yaml <<< 'EOF'
manifestVersion: 1.0
projects:
  - name: full-export
    path: projects/full-export
environmentGroups:
  - name: source
    environments:
      - name: source-tenant
        url:
          type: environment
          value: SOURCE_DT_URL
        auth:
          token:
            type: environment
            value: SOURCE_DT_TOKEN
EOF

# Set environment variables
export SOURCE_DT_URL="https://<source-tenant>.live.dynatrace.com"
export SOURCE_DT_TOKEN="dt0c01.xxxx..."

# Download all configuration
monaco download manifest.yaml --environment source-tenant

# Download specific schemas only
monaco download manifest.yaml --environment source-tenant \
  --specific-settings builtin:alerting.profile,builtin:monitoring.slo

# Download documents and automations
monaco download manifest.yaml --environment source-tenant \
  --only-documents --only-automations

```

Export Directory Structure

After download, your project looks like:

```
projects/full-export/  
├─ settings/  
│   ├── builtin_alerting_profile/  
│   ├── builtin_monitoring_slo/  
│   ├── builtin_kubernetes_metadata_enrichment/  
│   └─ ...  
├─ documents/  
│   ├── dashboards/  
│   └─ notebooks/  
├─ automations/  
│   └─ workflows/  
├─ bucket/  
└─ openpipeline/
```

4. Option A: Deploy Directly with Monaco

If your team does not use Terraform, deploy the exported configuration directly:

```
# Update manifest.yaml to point at the target tenant  
# Change environment URL and token to target values  
  
# Validate before deploying  
monaco validate manifest.yaml  
  
# Dry run – preview what will change  
monaco deploy manifest.yaml --environment target --dry-run  
  
# Deploy to target  
monaco deploy manifest.yaml --environment target
```

Note: Monaco resolves dependencies automatically. Buckets are created before OpenPipeline rules that reference them, enrichment rules before policies that depend on them.

5. Option B: Terraform Bootstrap

If your team uses Terraform, convert the Monaco export to HCL for ongoing management:

```
# Initialize Terraform with Dynatrace provider
cat &gt; providers.tf &lt;&lt; 'EOF'
terraform {
  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.91"
    }
  }
}

provider "dynatrace" {
  dt_env_url    = var.target_tenant_url
  dt_api_token  = var.target_api_token
  client_id     = var.target_client_id
  client_secret = var.target_client_secret
  account_id    = var.target_account_id
}
EOF

terraform init
```

Import Existing Resources

If the target tenant already has some configuration:

```
# Import existing resources into Terraform state
terraform import dynatrace_alerting.my_rule "&lt;alerting-profile-id&gt;"
terraform import dynatrace_slo_v2.my_slo "&lt;slo-id&gt;"

# Verify state matches
terraform plan
```

6. Export Validation

After export, validate completeness:

Check	Command	Expected
JSON validity	<code>monaco validate manifest.yaml</code>	0 errors
Schema coverage	<code>ls projects/full-export/settings/ \\ wc -l</code>	Matches API count
Document count	<code>ls projects/full-export/documents/ \\ wc -l</code>	Matches tenant count
No secrets in export	<code>grep -r "dt0c01" projects/</code>	0 matches

Next Steps

Continue to **S2S-03: IAM, SSO, and User Migration** to plan identity and access migration.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.